~\Desktop\preprocess.py

```python
from typing import Callable, Optional, Union
from uuid import uuid4

import anndata as ad
import lamindb as ln
import numpy as np
import pandas as pd
import scanpy as sc
from anndata import AnnData
from scipy.sparse import csr_matrix
from anndata import read_h5ad
from scdataloader import utils as data_utils
from upath import UPath

FULL_LENGTH_ASSAYS = [
    "EFO: 0700016",
    "EFO:0008930",
    "EFO:0008931",
]

MAXFILESIZE = 10_000_000_000


class Preprocessor:
    """
    Prepare data into training, valid and test split. Normalize raw expression
    values, binning or using other transform into the preset model input format.
    """

    def __init__(
        self,
        filter_gene_by_counts: Union[int, bool] = False,
        filter_cell_by_counts: Union[int, bool] = False,
        normalize_sum: float = 1e4,
        n_hvg_for_postp: int = 0,
        use_layer: Optional[str] = None,
        is_symbol: bool = False,
        hvg_flavor: str = "seurat_v3",
        binning: Optional[int] = None,
        result_binned_key: str = "X_binned",
        length_normalize: bool = False,
        force_preprocess: bool = False,
        min_dataset_size: int = 100,
        min_valid_genes_id: int = 10_000,
        min_nnz_genes: int = 200,
        maxdropamount: int = 50,
        madoutlier: int = 5,
        pct_mt_outlier: int = 8,
        batch_keys: list[str] = [
            "assay_ontology_term_id",
            "self_reported_ethnicity_ontology_term_id",
            "sex_ontology_term_id",
            "donor_id",
            "suspension_type",
        ],
        skip_validate: bool = False,
        additional_preprocess: Optional[Callable[[AnnData], AnnData]] = None,
        additional_postprocess: Optional[Callable[[AnnData], AnnData]] = None,
        do_postp: bool = True,
        organisms: list[str] = ["NCBITaxon:9606", "NCBITaxon:10090"],
        use_raw: bool = True,
    ) -> None:
        """
        Initializes the preprocessor and configures the workflow steps.

        Args:
            filter_gene_by_counts (int or bool, optional): Determines whether to filter genes by counts.
                If int, filters genes with counts. Defaults to False.
            filter_cell_by_counts (int or bool, optional): Determines whether to filter cells by counts.
                If int, filters cells with counts. Defaults to False.
            normalize_sum (float or bool, optional): Determines whether to normalize the total counts of each
to a specific value.
```

```
                Defaults to 1e4.
            log1p (bool, optional): Determines whether to apply log1p transform to the normalized data.
                Defaults to True.
            n_hvg_for_postp (int or bool, optional): Determines whether to subset to highly variable genes for the PCA.
                Defaults to False.
            hvg_flavor (str, optional): Specifies the flavor of highly variable genes selection.
                See :func:`scanpy.pp.highly_variable_genes` for more details. Defaults to "seurat_v3".
            binning (int, optional): Determines whether to bin the data into discrete values of number of bins provided.
            result_binned_key (str, optional): Specifies the key of :class:`~anndata.AnnData` to store the binned data.
                Defaults to "X_binned".
            length_normalize (bool, optional): Determines whether to length normalize the data.
                Defaults to False.
            force_preprocess (bool, optional): Determines whether to bypass the check of raw counts.
                Defaults to False.
            min_dataset_size (int, optional): The minimum size required for a dataset to be kept.
                Defaults to 100.
            min_valid_genes_id (int, optional): The minimum number of valid genes to keep a dataset.
                Defaults to 10_000.
            min_nnz_genes (int, optional): The minimum number of non-zero genes to keep a cell.
                Defaults to 200.
            maxdropamount (int, optional): The maximum amount of dropped cells per dataset. (2 for 50% drop, 3 for 33% drop, etc.)
                Defaults to 2.
            madoutlier (int, optional): The maximum absolute deviation of the outlier samples.
                Defaults to 5.
            pct_mt_outlier (int, optional): The maximum percentage of mitochondrial genes outlier.
                Defaults to 8.
            batch_key (str, optional): The key of :class:`~anndata.AnnData.obs` to use for batch information.
                This arg is used in the highly variable gene selection step.
            skip_validate (bool, optional): Determines whether to skip the validation step.
                Defaults to False.
        """
        self.filter_gene_by_counts = filter_gene_by_counts
        self.filter_cell_by_counts = filter_cell_by_counts
        self.normalize_sum = normalize_sum
        self.hvg_flavor = hvg_flavor
        self.binning = binning
        self.organisms = organisms
        self.result_binned_key = result_binned_key
        self.additional_preprocess = additional_preprocess
        self.additional_postprocess = additional_postprocess
        self.force_preprocess = force_preprocess
        self.min_dataset_size = min_dataset_size
        self.min_valid_genes_id = min_valid_genes_id
        self.min_nnz_genes = min_nnz_genes
        self.maxdropamount = maxdropamount
        self.madoutlier = madoutlier
        self.n_hvg_for_postp = n_hvg_for_postp
        self.pct_mt_outlier = pct_mt_outlier
        self.batch_keys = batch_keys
        self.length_normalize = length_normalize
        self.skip_validate = skip_validate
        self.use_layer = use_layer
        self.is_symbol = is_symbol
        self.do_postp = do_postp
        self.use_raw = use_raw

    def __call__(self, adata, dataset_id=None) -> AnnData:
        if adata[0].obs.organism_ontology_term_id.iloc[0] not in self.organisms:
            raise ValueError(
                "we cannot work with this organism",
                adata[0].obs.organism_ontology_term_id.iloc[0],
            )
        if self.additional_preprocess is not None:
            adata = self.additional_preprocess(adata)
        if adata.raw is not None and self.use_raw:
            adata.X = adata.raw.X
            del adata.raw
        if self.use_layer is not None:
            adata.X = adata.layers[self.use_layer]
        if adata.layers is not None:
            if "counts" in adata.layers.keys():
                if np.abs(adata[:50_000].X.astype(int) - adata[:50_000].X).sum():
```

```
                print("X was not raw counts, using 'counts' layer")
                adata.X = adata.layers["counts"].copy()
            print("Dropping layers: ", adata.layers.keys())
            del adata.layers
        if len(adata.varm.keys()) > 0:
            del adata.varm
        if len(adata.obsm.keys()) > 0 and self.do_postp:
            del adata.obsm
        if len(adata.obsp.keys()) > 0 and self.do_postp:
            del adata.obsp
        # check that it is a count
        print("checking raw counts")
        if np.abs(
            adata[:50_000].X.astype(int) - adata[:50_000].X
        ).sum():  # check if likely raw data
            if not self.force_preprocess:
                raise ValueError(
                    "Data is not raw counts, please check layers, find raw data, or bypass with force_preprocess"
                )
            else:
                print(
                    "Data is not raw counts, please check layers, find raw data, or bypass with force_preprocess"
                )
            # please check layers
            # if not available count drop
        prevsize = adata.shape[0]
        # dropping non primary
        if "is_primary_data" in adata.obs.columns:
            adata = adata[adata.obs.is_primary_data]
        if adata.shape[0] < self.min_dataset_size:
            raise Exception("Dataset dropped due to too many secondary cells")
        print(
            "removed {} non primary cells, {} renamining".format(
                prevsize - adata.shape[0], adata.shape[0]
            )
        )
        # # cleanup and dropping low expressed genes and unexpressed cells
        prevsize = adata.shape[0]
        adata.obs["nnz"] = np.array(np.sum(adata.X != 0, axis=1).flatten())[0]
        if self.filter_gene_by_counts:
            sc.pp.filter_genes(adata, min_counts=self.filter_gene_by_counts)
        if self.filter_cell_by_counts:
            sc.pp.filter_cells(
                adata,
                min_counts=self.filter_cell_by_counts,
            )
        if self.min_nnz_genes:
            sc.pp.filter_cells(
                adata,
                min_genes=self.min_nnz_genes,
            )
        # if lost > 50% of the dataset, drop dataset
        # load the genes
        genesdf = data_utils.load_genes(adata.obs.organism_ontology_term_id.iloc[0])

        if prevsize / adata.shape[0] > self.maxdropamount:
            raise Exception(
                "Dataset dropped due to low expressed genes and unexpressed cells: factor of "
                + str(prevsize / adata.shape[0])
            )
        if adata.shape[0] < self.min_dataset_size:
            raise Exception(
                "Dataset dropped due to low expressed genes and unexpressed cells: current size: "
                + str(adata.shape[0])
            )
        print(
            "filtered out {} cells, {} renamining".format(
                prevsize - adata.shape[0], adata.shape[0]
            )
        )

        if self.is_symbol or not adata.var.index.str.contains("ENS").any():
            if not adata.var.index.str.contains("ENS").any():
                print("No ENS genes found, assuming gene symbols...")
            genesdf["ensembl_gene_id"] = genesdf.index
```

```
            var = (
                adata.var.merge(
                    genesdf.drop_duplicates("symbol").set_index("symbol", drop=False),
                    left_index=True,
                    right_index=True,
                    how="inner",
                )
                .sort_values(by="ensembl_gene_id")
                .set_index("ensembl_gene_id")
            )
            adata = adata[:, var["symbol"]]
            adata.var = var
            genesdf = genesdf.set_index("ensembl_gene_id")

        intersect_genes = set(adata.var.index).intersection(set(genesdf.index))
        print(f"Removed {len(adata.var.index) - len(intersect_genes)} genes.")
        if len(intersect_genes) < self.min_valid_genes_id:
            raise Exception("Dataset dropped due to too many genes not mapping to it")
        adata = adata[:, list(intersect_genes)]
        # marking unseen genes
        unseen = set(genesdf.index) - set(adata.var.index)
        # adding them to adata
        emptyda = ad.AnnData(
            csr_matrix((adata.shape[0], len(unseen)), dtype=np.float32),
            var=pd.DataFrame(index=list(unseen)),
            obs=pd.DataFrame(index=adata.obs.index),
        )
        adata = ad.concat([adata, emptyda], axis=1, join="outer", merge="only")
        # do a validation function
        adata.uns["unseen_genes"] = list(unseen)
        if dataset_id is not None:
            adata.uns["dataset_id"] = dataset_id
        if not self.skip_validate:
            print("validating")
            data_utils.validate(
                adata, organism=adata.obs.organism_ontology_term_id[0], need_all=False
            )
            # length normalization
            if (
                adata.obs["assay_ontology_term_id"].isin(FULL_LENGTH_ASSAYS).any()
                and self.length_normalize
            ):
                print("doing length norm")
                subadata = data_utils.length_normalize(
                    adata[adata.obs["assay_ontology_term_id"].isin(FULL_LENGTH_ASSAYS)],
                )

                adata = ad.concat(
                    [
                        adata[
                            ~adata.obs["assay_ontology_term_id"].isin(
                                FULL_LENGTH_ASSAYS
                            )
                        ],
                        subadata,
                    ],
                    axis=0,
                    join="outer",
                    merge="only",
                )

        # QC

        adata.var[genesdf.columns] = genesdf.loc[adata.var.index]
        print("startin QC")
        sc.pp.calculate_qc_metrics(
            adata, qc_vars=["mt", "ribo", "hb"], inplace=True, percent_top=[20]
        )

        adata.obs["outlier"] = (
            data_utils.is_outlier(adata, "total_counts", self.madoutlier)
            | data_utils.is_outlier(adata, "n_genes_by_counts", self.madoutlier)
            | data_utils.is_outlier(
                adata, "pct_counts_in_top_20_genes", self.madoutlier
            )
```

```
        )

        adata.obs["mt_outlier"] = data_utils.is_outlier(adata, "pct_counts_mt", 3) | (
            adata.obs["pct_counts_mt"] > self.pct_mt_outlier
        )
        total_outliers = (adata.obs["outlier"] | adata.obs["mt_outlier"]).sum()
        total_cells = adata.shape[0]
        percentage_outliers = (total_outliers / total_cells) * 100
        print(
            f"Seeing {total_outliers} outliers ({percentage_outliers:.2f}% of total dataset):"
        )
        # if percentage_outliers > 50:
        #    raise Exception("More than 50% of the dataset has been dropped due to outliers.")
        # adata = adata[(~adata.obs.outlier) & (~adata.obs.mt_outlier)].copy()
        # remaining

        # based on the topometry paper https://www.biorxiv.org/content/10.1101/2022.03.14.484134v2
        # https://rapids-singlecell.readthedocs.io/en/latest/api/generated/rapids_singlecell.pp.pca.html#rapids_singlecell.pp.pca
        if self.do_postp:
            print("normalize")
            adata.layers["norm"] = sc.pp.log1p(
                sc.pp.normalize_total(
                    adata, target_sum=self.normalize_sum, inplace=False
                )["X"]
            )
            # step 5: subset hvg
            batches = []
            for i in self.batch_keys:
                if i in adata.obs.columns:
                    batches.append(i)
            adata.obs["batches"] = adata.obs[batches].apply(
                lambda x: ",".join(x.dropna().astype(str)), axis=1
            )
            if self.n_hvg_for_postp:
                try:
                    sc.pp.highly_variable_genes(
                        adata,
                        n_top_genes=self.n_hvg_for_postp,
                        batch_key="batches",
                        flavor=self.hvg_flavor,
                        subset=False,
                        layer="norm",
                    )
                except (ValueError, ZeroDivisionError) as e:
                    print("retrying with span")
                    sc.pp.highly_variable_genes(
                        adata,
                        n_top_genes=self.n_hvg_for_postp,
                        # batch_key="batches",
                        flavor=self.hvg_flavor,
                        span=0.5,
                        subset=False,
                        layer="norm",
                    )

            adata.obsm["X_pca"] = sc.pp.pca(
                adata.layers["norm"][:, adata.var.highly_variable]
                if "highly_variable" in adata.var.columns
                else adata.layers["norm"],
                n_comps=200 if adata.shape[0] > 200 else adata.shape[0] - 2,
            )

            # additional
            if self.additional_postprocess is not None:
                adata = self.additional_postprocess(adata)
        adata = adata[:, adata.var.sort_index().index]
        # create random ids for all cells
        adata.obs.index = [str(uuid4()) for _ in range(adata.shape[0])]
        # not necessary, int causes issues in some cases and you
        # do not get more information / less space for your bucks
        # adata.X = adata.X.astype(int32)
        # step 6: binning
        if self.binning:
            print("Binning data ...")
```

```python
            if not isinstance(self.binning, int):
                raise ValueError(
                    "Binning arg must be an integer, but got {}.".format(self.binning)
                )
            # NOTE: the first bin is always a spectial for zero
            n_bins = self.binning
            binned_rows = []
            bin_edges = []

            if adata.X.min() < 0:
                raise ValueError(
                    f"Assuming non-negative data, but got min value {adata.X.min()}."
                )
            for row in adata.X:
                if row.max() == 0:
                    print(
                        "The input data contains all zero rows. Please make sure "
                        "this is expected. You can use the `filter_cell_by_counts` "
                        "arg to filter out all zero rows."
                    )
                    binned_rows.append(np.zeros_like(row, dtype=np.int64))
                    bin_edges.append(np.array([0] * n_bins))
                    continue
                non_zero_ids = row.nonzero()
                non_zero_row = row[non_zero_ids]
                bins = np.quantile(non_zero_row, np.linspace(0, 1, n_bins - 1))
                # bins = np.sort(np.unique(bins))
                # NOTE: comment this line for now, since this will make the each category
                # has different relative meaning across datasets
                non_zero_digits = _digitize(non_zero_row, bins)
                assert non_zero_digits.min() >= 1
                assert non_zero_digits.max() <= n_bins - 1
                binned_row = np.zeros_like(row, dtype=np.int64)
                binned_row[non_zero_ids] = non_zero_digits
                binned_rows.append(binned_row)
                bin_edges.append(np.concatenate([[0], bins]))
            adata.layers[self.result_binned_key] = np.stack(binned_rows)
            adata.obsm["bin_edges"] = np.stack(bin_edges)
        print("done")
        print(adata)
        return adata


class LaminPreprocessor(Preprocessor):
    def __init__(
        self,
        *args,
        cache: bool = True,
        keep_files: bool = True,
        force_preloaded: bool = False,
        **kwargs,
    ):
        super().__init__(*args, **kwargs)
        self.cache = cache
        self.keep_files = keep_files
        self.force_preloaded = force_preloaded

    def __call__(
        self,
        data: Union[ln.Collection, AnnData] = None,
        name: str = "preprocessed dataset",
        description: str = "preprocessed dataset using scprint",
        start_at: int = 0,
        version: str = "2",
    ):
        """
        format controls the different input value wrapping, including categorical
        binned style, fixed-sum normalized counts, log1p fixed-sum normalized counts, etc.

        Args:
            adata (AnnData): The AnnData object to preprocess.
            batch_key (str, optional): The key of AnnData.obs to use for batch information. This arg
                is used in the highly variable gene selection step.
        """
        files = []
```

```
        all_ready_processed_keys = set()
        if self.cache:
            for i in ln.Artifact.filter(description=description):
                all_ready_processed_keys.add(i.stem_uid)
        if isinstance(data, AnnData):
            return super().__call__(data)
        elif isinstance(data, ln.Collection):
            for i, file in enumerate(data.artifacts.all()[start_at:]):
                # use the counts matrix
                i = i + start_at
                print(i)
                if file.stem_uid in all_ready_processed_keys:
                    print(f"{file.stem_uid} is already processed... not preprocessing")
                    continue
                print(file)

                path = cache_path(file) if self.force_preloaded else file.cache()
                backed = read_h5ad(path, backed="r")
                if backed.obs.is_primary_data.sum() == 0:
                    print(f"{file.key} only contains non primary cells.. dropping")
                    # Save the stem_uid to a file to avoid loading it again
                    with open("nonprimary.txt", "a") as f:
                        f.write(f"{file.stem_uid}\n")
                    continue
                if backed.shape[1] < 1000:
                    print(
                        f"{file.key} only contains less than 1000 genes and is likely not scRNAseq... dropping"
                    )
                    continue
                if file.size <= MAXFILESIZE:
                    adata = backed.to_memory()
                    print(adata)
                else:
                    badata = backed
                    print(badata)
                try:
                    if file.size > MAXFILESIZE:
                        print(
                            f"dividing the dataset as it is too large: {file.size//1_000_000_000}Gb"
                        )
                        num_blocks = int(np.ceil(file.size / (MAXFILESIZE / 2)))
                        block_size = int(
                            (np.ceil(badata.shape[0] / 30_000) * 30_000) // num_blocks
                        )
                        print("num blocks ", num_blocks)
                        for j in range(num_blocks):
                            if j == 0 and i == 390:
                                continue
                            start_index = j * block_size
                            end_index = min((j + 1) * block_size, badata.shape[0])
                            block = badata[start_index:end_index].to_memory()
                            print(block)
                            block = super().__call__(
                                block, dataset_id=file.stem_uid + "_p" + str(j)
                            )
                            myfile = ln.Artifact.from_anndata(
                                block,
                                description=description
                                + " n"
                                + str(i)
                                + " p"
                                + str(j)
                                + " ( revises file "
                                + str(file.key)
                                + " )",
                                version=version,
                            )
                            myfile.save()
                            if self.keep_files:
                                files.append(myfile)
                            else:
                                del myfile
                                del block

                    else:
```

```
                        adata = super().__call__(adata, dataset_id=file.stem_uid)
                        myfile = ln.Artifact.from_anndata(
                            adata,
                            revises=file,
                            description=description + " p" + str(i),
                            version=version,
                        )
                        myfile.save()
                        if self.keep_files:
                            files.append(myfile)
                        else:
                            del myfile
                            del adata

                except ValueError as v:
                    if v.args[0].startswith("we cannot work with this organism"):
                        print(v)
                        continue
                    else:
                        raise v
                except Exception as e:
                    if e.args[0].startswith("Dataset dropped due to"):
                        print(e)
                        continue
                    else:
                        raise e

                # issues with KLlggfw6I6lvmbqiZm46
            if self.keep_files:
                dataset = ln.Collection(files, name=name, description=description)
                dataset.save()
                return dataset
            else:
                return
        else:
            raise ValueError("Please provide either anndata or ln.Collection")


def is_log1p(adata: AnnData) -> bool:
    """
    Check if the data is already log1p transformed.

    Args:

    adata (:class:`AnnData`):
        The :class:`AnnData` object to preprocess.
    obs_key (:class:`str`, optional):
        The key of :class:`AnnData.obs` to use for batch information. This arg
        is used in the highly variable gene selection step.
    """
    max_, min_ = adata.X.max(), adata.X.min()
    if max_ > 30:
        return False
    if min_ < 0:
        return False

    non_zero_min = adata.X[adata.X > 0].min()
    if non_zero_min >= 1:
        return False

    return True


def _digitize(x: np.ndarray, bins: np.ndarray, side="both") -> np.ndarray:
    """
    Digitize the data into bins. This method spreads data uniformly when bins
    have same values.

    Args:

    x (:class:`np.ndarray`):
        The data to digitize.
    bins (:class:`np.ndarray`):
        The bins to use for digitization, in increasing order.
    side (:class:`str`, optional):
```

```
        The side to use for digitization. If "one", the left side is used. If
        "both", the left and right side are used. Default to "one".

    Returns:

    :class:`np.ndarray`:
        The digitized data.
    """
    assert x.ndim == 1 and bins.ndim == 1

    left_digits = np.digitize(x, bins)
    if side == "one":
        return left_digits

    right_difits = np.digitize(x, bins, right=True)

    rands = np.random.rand(len(x))  # uniform random numbers

    digits = rands * (right_difits - left_digits) + left_digits
    digits = np.ceil(digits).astype(np.int64)
    return digits


def binning(row: np.ndarray, n_bins: int) -> np.ndarray:
    """Binning the row into n_bins."""
    # TODO: use torch.quantile and torch.bucketize
    dtype = row.dtype
    if row.min() <= 0:
        non_zero_ids = row.nonzero()
        non_zero_row = row[non_zero_ids]
        bins = np.quantile(non_zero_row, np.linspace(0, 1, n_bins - 1))
        non_zero_digits = _digitize(non_zero_row, bins)
        binned_row = np.zeros_like(row, dtype=np.int64)
        binned_row[non_zero_ids] = non_zero_digits
    else:
        bins = np.quantile(row, np.linspace(0, 1, n_bins - 1))
        binned_row = _digitize(row, bins)
    return binned_row.astype(dtype)


#################
### specific #####
#################


def additional_preprocess(adata):
    adata.obs = adata.obs.replace(
        {
            "self_reported_ethnicity_ontology_term_id": {
                "multiethnic": "unknown",
                "American": "unknown",
                "Jewish Israeli": "unknown",
                "na": "unknown",
            }
        }
    )  # multi ethnic will have to get renamed
    adata.obs["cell_culture"] = False
    # if cell_type contains the word "(cell culture)" then it is a cell culture and we mark it as so and remove this from the cell type
    loc = adata.obs["cell_type_ontology_term_id"].str.contains(
        "(cell culture)", regex=False
    )
    if loc.sum() > 0:
        adata.obs["cell_type_ontology_term_id"] = adata.obs[
            "cell_type_ontology_term_id"
        ].astype(str)
        adata.obs.loc[loc, "cell_culture"] = True
        adata.obs.loc[loc, "cell_type_ontology_term_id"] = adata.obs.loc[
            loc, "cell_type_ontology_term_id"
        ].str.replace(" (cell culture)", "")

    loc = adata.obs["tissue_ontology_term_id"].str.contains(
        "(cell culture)", regex=False
    )
    if loc.sum() > 0:
```

```python
        adata.obs.loc[loc, "cell_culture"] = True
        adata.obs["tissue_ontology_term_id"] = adata.obs[
            "tissue_ontology_term_id"
        ].astype(str)
        adata.obs.loc[loc, "tissue_ontology_term_id"] = adata.obs.loc[
            loc, "tissue_ontology_term_id"
        ].str.replace(" (cell culture)", "")

    loc = adata.obs["tissue_ontology_term_id"].str.contains("(organoid)", regex=False)
    if loc.sum() > 0:
        adata.obs.loc[loc, "cell_culture"] = True
        adata.obs["tissue_ontology_term_id"] = adata.obs[
            "tissue_ontology_term_id"
        ].astype(str)
        adata.obs.loc[loc, "tissue_ontology_term_id"] = adata.obs.loc[
            loc, "tissue_ontology_term_id"
        ].str.replace(" (organoid)", "")

    loc = adata.obs["tissue_ontology_term_id"].str.contains("CL:", regex=False)
    if loc.sum() > 0:
        adata.obs["tissue_ontology_term_id"] = adata.obs[
            "tissue_ontology_term_id"
        ].astype(str)
        adata.obs.loc[loc, "tissue_ontology_term_id"] = "unknown"
    return adata


def additional_postprocess(adata):
    # import palantir

    # define the "up to" 10 neighbors for each cells and add to obs
    # compute neighbors
    # need to be connectivities and same labels [cell type, assay, dataset, disease]
    # define the "neighbor" up to 10(N) cells and add to obs
    # define the "next time point" up to 5(M) cells and add to obs  # step 1: filter genes
    # if len(adata.obs["batches"].unique()) > 1:
    #    sc.external.pp.harmony_integrate(adata, key="batches")
    #    sc.pp.neighbors(adata, use_rep="X_pca_harmony")
    # else:
    sc.pp.neighbors(adata, use_rep="X_pca")
    sc.tl.leiden(adata, key_added="leiden_2", resolution=2.0)
    sc.tl.leiden(adata, key_added="leiden_1", resolution=1.0)
    sc.tl.leiden(adata, key_added="leiden_0.5", resolution=0.5)
    sc.tl.umap(adata)
    mid = adata.uns["dataset_id"] if "dataset_id" in adata.uns else "unknown_id"
    sc.pl.umap(
        adata,
        ncols=1,
        color=["cell_type", "batches"],
        save="_" + mid + ".png",
    )
    COL = "cell_type_ontology_term_id"
    NEWOBS = "clust_cell_type"
    MINCELLS = 10
    MAXSIM = 0.94
    from collections import Counter

    from .config import MAIN_HUMAN_MOUSE_DEV_STAGE_MAP

    adata.obs[NEWOBS] = (
        adata.obs[COL].astype(str) + "_" + adata.obs["leiden_1"].astype(str)
    )
    coun = Counter(adata.obs[NEWOBS])
    relab = {}
    for i in adata.obs[COL].unique():
        num = 0
        for n, c in sorted(coun.items(), key=lambda x: x[1], reverse=True):
            if i in n:
                if c < MINCELLS or num == 0:
                    relab[n] = i
                else:
                    relab[n] = i + "_" + str(num)
                num += 1

    adata.obs[NEWOBS] = adata.obs[NEWOBS].map(relab)
```

```python

    cluster_means = pd.DataFrame(
        np.array(
            [
                adata.X[adata.obs[NEWOBS] == i].mean(axis=0)
                for i in adata.obs[NEWOBS].unique()
            ]
        )[:, 0, :],
        index=adata.obs[NEWOBS].unique(),
    )

    # Calculate correlation matrix between clusters
    cluster_similarity = cluster_means.T.corr()
    cluster_similarity.values[np.tril_indices(len(cluster_similarity), -1)] = 0

    # Get pairs with similarity > 0.95
    high_sim_pairs = []
    for i in range(len(cluster_similarity)):
        for j in range(i + 1, len(cluster_similarity)):
            if (
                cluster_similarity.iloc[i, j] > MAXSIM
                and cluster_similarity.columns[i].split("_")[0]
                == cluster_similarity.columns[j].split("_")[0]
            ):
                high_sim_pairs.append(
                    (
                        cluster_similarity.index[i],
                        cluster_similarity.columns[j],
                    )
                )
    # Create mapping for merging similar clusters
    merge_mapping = {}
    for pair in high_sim_pairs:
        if pair[0] not in merge_mapping:
            merge_mapping[pair[1]] = pair[0]
        else:
            merge_mapping[pair[1]] = merge_mapping[pair[0]]

    # Apply merging
    adata.obs[NEWOBS] = adata.obs[NEWOBS].map(merge_mapping).fillna(adata.obs[NEWOBS])
    adata.obs[NEWOBS] = adata.obs[NEWOBS].astype(str)
    coun = Counter(adata.obs[NEWOBS]).most_common()
    merge_mapping = {}
    for i in adata.obs[COL].unique():
        num = 0
        for j, c in coun:
            if i in j:
                merge_mapping[j] = i + "_" + str(num) if num > 0 else i
                num += 1
    adata.obs[NEWOBS] = adata.obs[NEWOBS].map(merge_mapping).fillna(adata.obs[NEWOBS])

    import bionty as bt

    stages = adata.obs["development_stage_ontology_term_id"].unique()
    if adata.obs.organism_ontology_term_id.unique() == ["NCBITaxon:9606"]:
        relabel = {i: i for i in stages}
        for stage in stages:
            stage_obj = bt.DevelopmentalStage.filter(ontology_id=stage).first()
            parents = set([i.ontology_id for i in stage_obj.parents.filter()])
            parents = parents - set(
                [
                    "HsapDv:0010000",
                    "HsapDv:0000204",
                    "HsapDv:0000227",
                ]
            )
            if len(parents) > 0:
                for p in parents:
                    if p in MAIN_HUMAN_MOUSE_DEV_STAGE_MAP:
                        relabel[stage] = p
        adata.obs["simplified_dev_stage"] = adata.obs[
            "development_stage_ontology_term_id"
        ].map(relabel)
    elif adata.obs.organism_ontology_term_id.unique() == ["NCBITaxon:10090"]:
        rename_mapping = {
```

```
            k: v for v, j in MAIN_HUMAN_MOUSE_DEV_STAGE_MAP.items() for k in j
        }
        relabel = {i: "unknown" for i in stages}
        for stage in stages:
            if stage in rename_mapping:
                relabel[stage] = rename_mapping[stage]
        adata.obs["simplified_dev_stage"] = adata.obs[
            "development_stage_ontology_term_id"
        ].map(relabel)
    else:
        raise ValueError("organism not supported")
    # palantir.utils.run_diffusion_maps(adata, n_components=20)
    # palantir.utils.determine_multiscale_space(adata)
    # terminal_states = palantir.utils.find_terminal_states(
    #    adata,
    #    celltypes=adata.obs.cell_type_ontology_term_id.unique(),
    #    celltype_column="cell_type_ontology_term_id",
    # )
    # sc.tl.diffmap(adata)
    # adata.obs["heat_diff"] = 1
    # for terminal_state in terminal_states.index.tolist():
    #    adata.uns["iroot"] = np.where(adata.obs.index == terminal_state)[0][0]
    #    sc.tl.dpt(adata)
    #    adata.obs["heat_diff"] = np.minimum(
    #        adata.obs["heat_diff"], adata.obs["dpt_pseudotime"]
    #    )
    return adata


def cache_path(artifact):
    cloud_path = UPath(artifact.storage.root) / artifact.key
    cache_path = ln.setup.settings.paths.cloud_to_local_no_update(cloud_path)
    return cache_path

```